

Report on Assignment 1. Digital Signal Processing

Course: Speech Recognition and Generation course

Assignment: Speech Recognition — Binary YES/NO Classifier

Author: Natalya Muravyova

University: ITMO, AITH

Date: March 2025

1. Introduction

In this project, I focus on binary classification of spoken words "yes" and "no" from the [SpeechCommands dataset](#). The goal is to compare model performance and computational efficiency under different configurations of:

- Number of Mel filter banks (`n_mels`)
- Grouped convolutions (`groups`) in 1D CNN

I preprocess the waveforms using a custom `LogMelFilterBanks` layer and evaluate the effect of architectural choices using loss, accuracy, parameters, and FLOPs.

2. Dataset Preparation

I used the built-in `SPEECHCOMMANDS` dataset from `torchaudio.datasets`. The dataset comes with predefined training, validation, and test splits.

After loading, I filtered the dataset to retain only the samples labeled "yes" or "no", forming new filtered subsets:

This gave me the binary dataset used throughout the experiments.

To use the `LogMelFilterBanks` features in the training pipeline, I created a custom dataset class `YesNoDataset`. This class transforms raw waveforms into log-mel filterbank features and maps labels to binary targets (1 for "yes", 0 for "no").

This setup allowed training a CNN model on consistent, preprocessed spectrogram features, using the `LogMelFilterBanks` layer.

3. Feature Extraction

3.1. `LogMelFilterBanks` Layer

In this section, I implemented a custom PyTorch layer `LogMelFilterBanks` for extracting logarithms of Mel-scale Filterbank energies using only basic Torch operations, as required.

The class `LogMelFilterBanks(nn.Module)` consists of the following key components:

- `__init__()`: Initializes STFT and Mel filterbank parameters, including FFT size, hop length, sample rate, window function, number of Mel filters, and others. The Hann window is used as specified.
- `_init_melscale_fbanks()`: Generates the Mel filterbank matrix using `torchaudio.functional.melscale_fbanks()` with proper parameters.
- `spectrogram()`: Computes the power spectrogram from raw audio using `torch.stft()`, followed by element-wise absolute value and power operation.
- `forward()`: Multiplies the spectrogram with the Mel filterbank matrix and applies a logarithm with a small constant added for numerical stability.

The final output of the layer has the shape `(batch, n_mels, n_frames)` and matches the output of `torchaudio.transforms.MelSpectrogram` with log transformation applied.

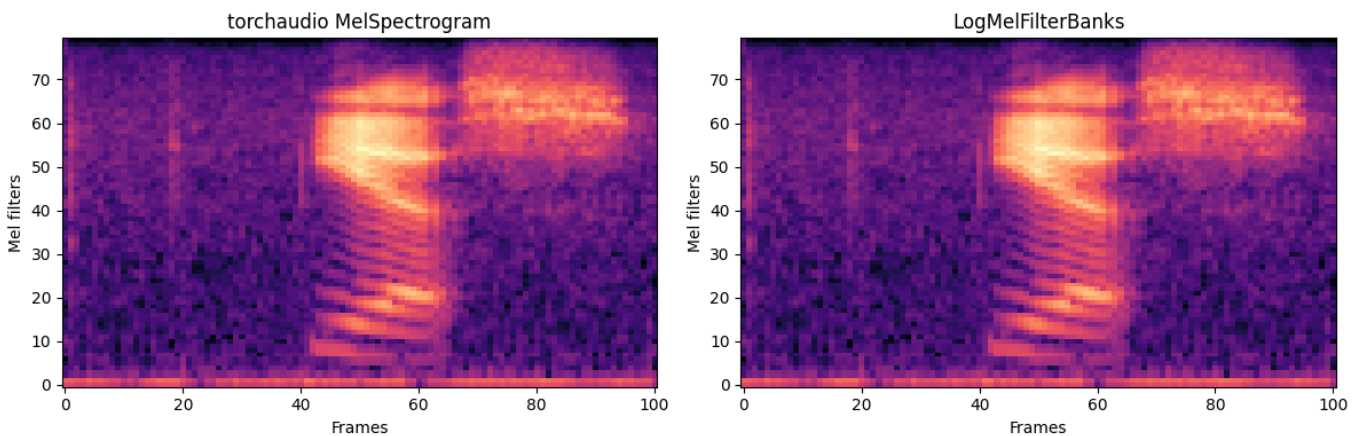
3.2. Comparison with torchaudio

To validate the implementation, I compared the output of the custom `LogMelFilterBanks` layer against `torchaudio.transforms.MelSpectrogram` followed by `torch.log`.

Both outputs were computed from the same input signal and the assertions passed.

Then, both results were plotted side-by-side for visual comparison:

- **Left:** torchaudio + log
- **Right:** Custom `LogMelFilterBanks`



The spectrograms appear nearly identical, confirming the correctness of the implementation. Both clearly capture time-frequency patterns across Mel filters, with the same structure and energy distribution.

4. Model

4.1. Model Architecture

For the classification task, I implemented a simple CNN (`SimpleCNN`) using `torch.nn.Conv1d`. The model processes log-mel filterbank features produced by the custom `LogMelFilterBanks` layer.

The architecture is as follows:

- `Conv1d(n_mels, 128, kernel_size=5, groups=groups)`: a grouped 1D convolution layer.

- `ReLU` activation.
 - `AdaptiveAvgPool1d(1)`: to reduce temporal dimension to 1.
 - `Linear(128, 1)`: a fully connected layer for binary classification.
-

4.2. Training Setup

The model was trained using the following configuration:

- **Loss function:** Binary Cross Entropy with Logits (`BCEWithLogitsLoss`)
- **Optimizer:** Adam (learning rate = 0.001)
- **Batch size:** 32
- **Epochs:** 10

The training loop tracks:

- Training and validation loss per epoch
- Accuracy per epoch
- Training time per experiment

Helper functions `train_one_epoch()` and `evaluate()` encapsulate training and validation logic respectively. Binary classification outputs were thresholded using `sigmoid(logits) > 0.5`.

Additionally, the number of model parameters and FLOPs were computed using the `get_model_complexity_info` function.

5. Experiments

5.1. Experiments with `n_mels`

To explore the impact of the number of Mel filters, I ran experiments with the following values:

- `n_mels` = 20
- `n_mels` = 40
- `n_mels` = 80

Each experiment involved:

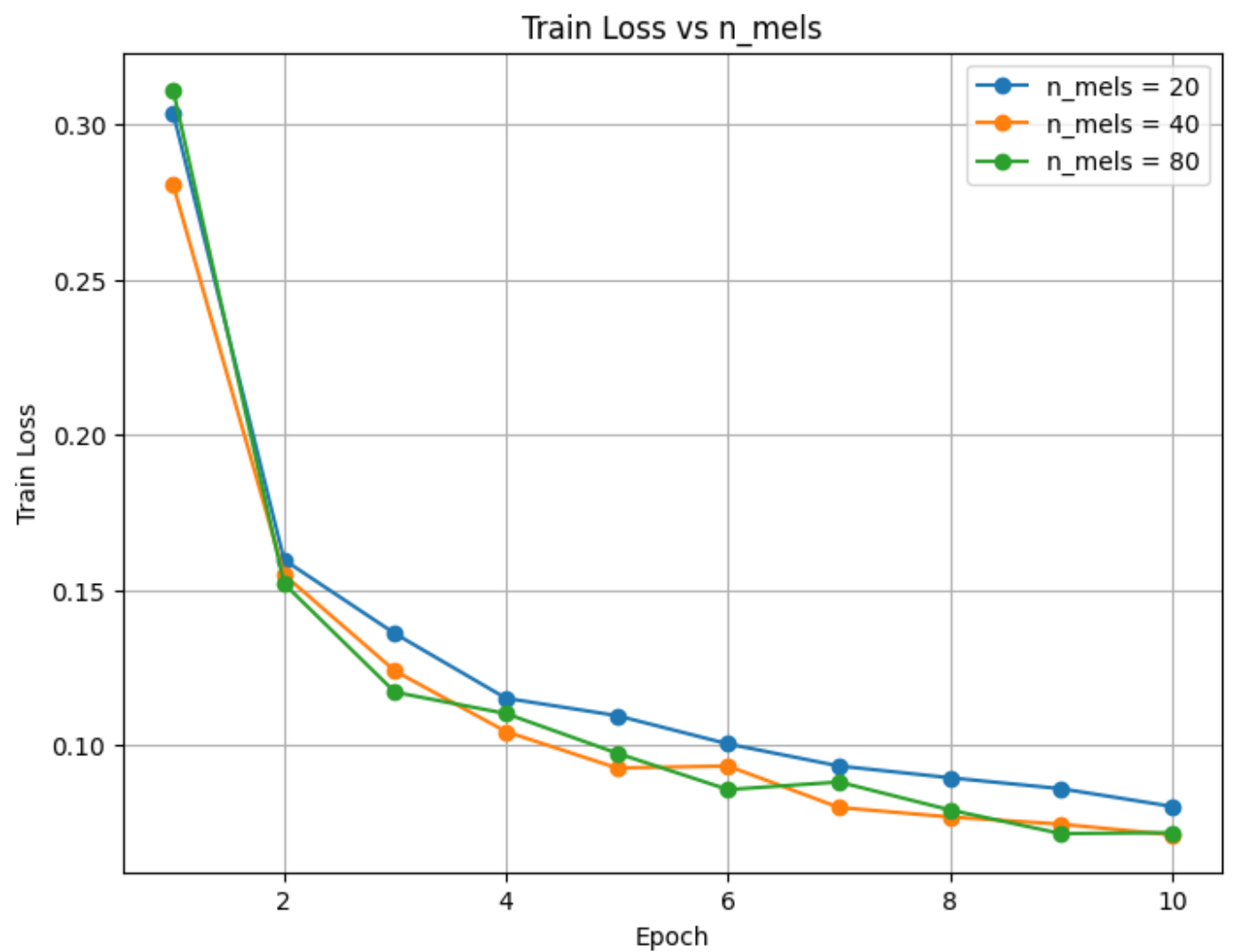
- Initializing the `LogMelFilterBanks` layer with the specified number of filters
- Training the `SimpleCNN` model for 10 epochs
- Logging training/validation loss and accuracy
- Measuring training time, number of parameters, and FLOPs

Results were stored in dictionaries for further analysis and visualization

5.1.2. Graphs and Observations

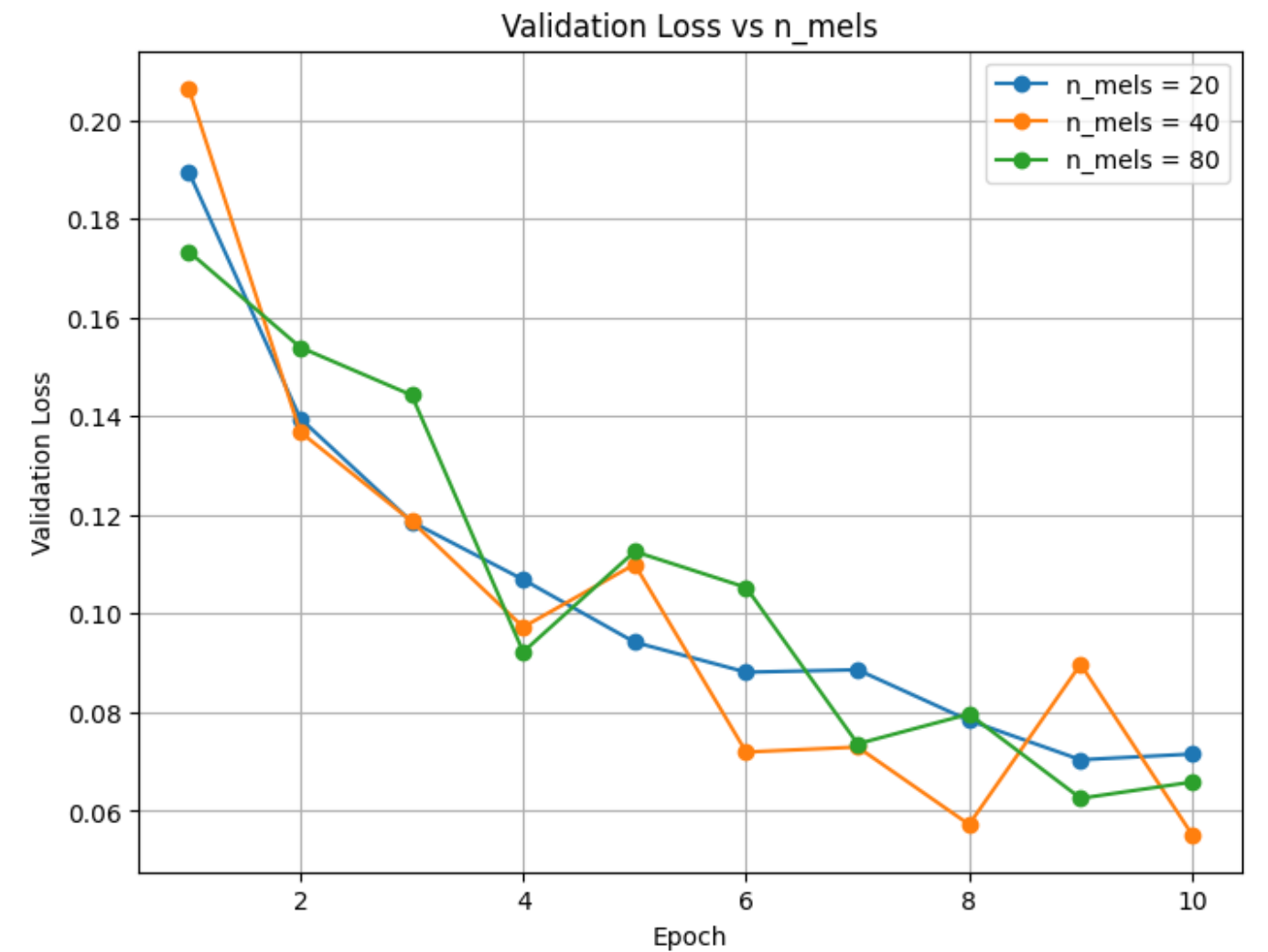
To better understand the impact of `n_mels` on model performance and training dynamics, I plotted key metrics across 10 training epochs. The following graphs summarize the results:

Training Loss vs `n_mels`



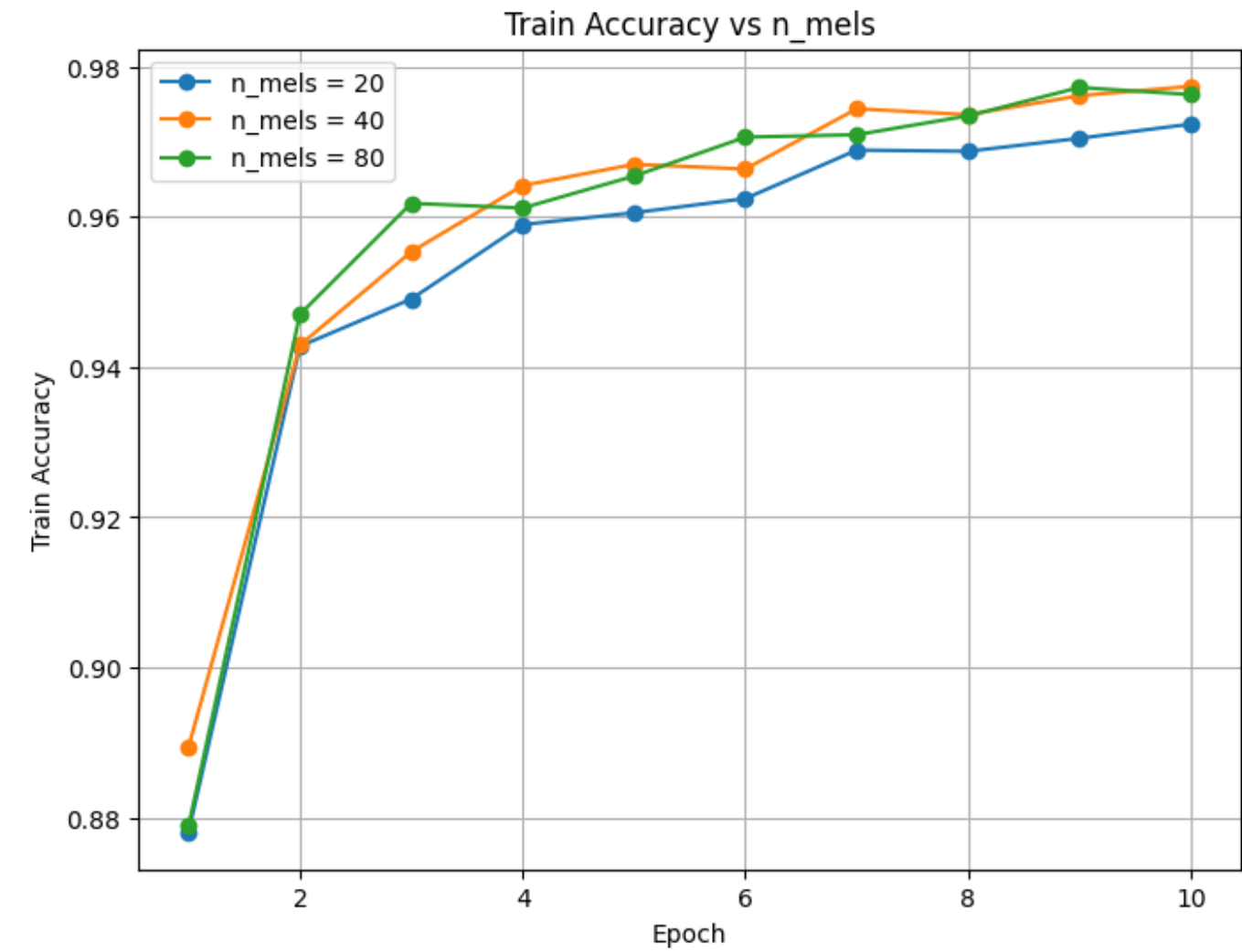
As the number of Mel filterbanks increases, the training loss decreases. The drop is especially noticeable between 20 and 40 Mel banks. After 40, further reduction becomes marginal, suggesting diminishing returns.

Validation Loss vs `n_mels`



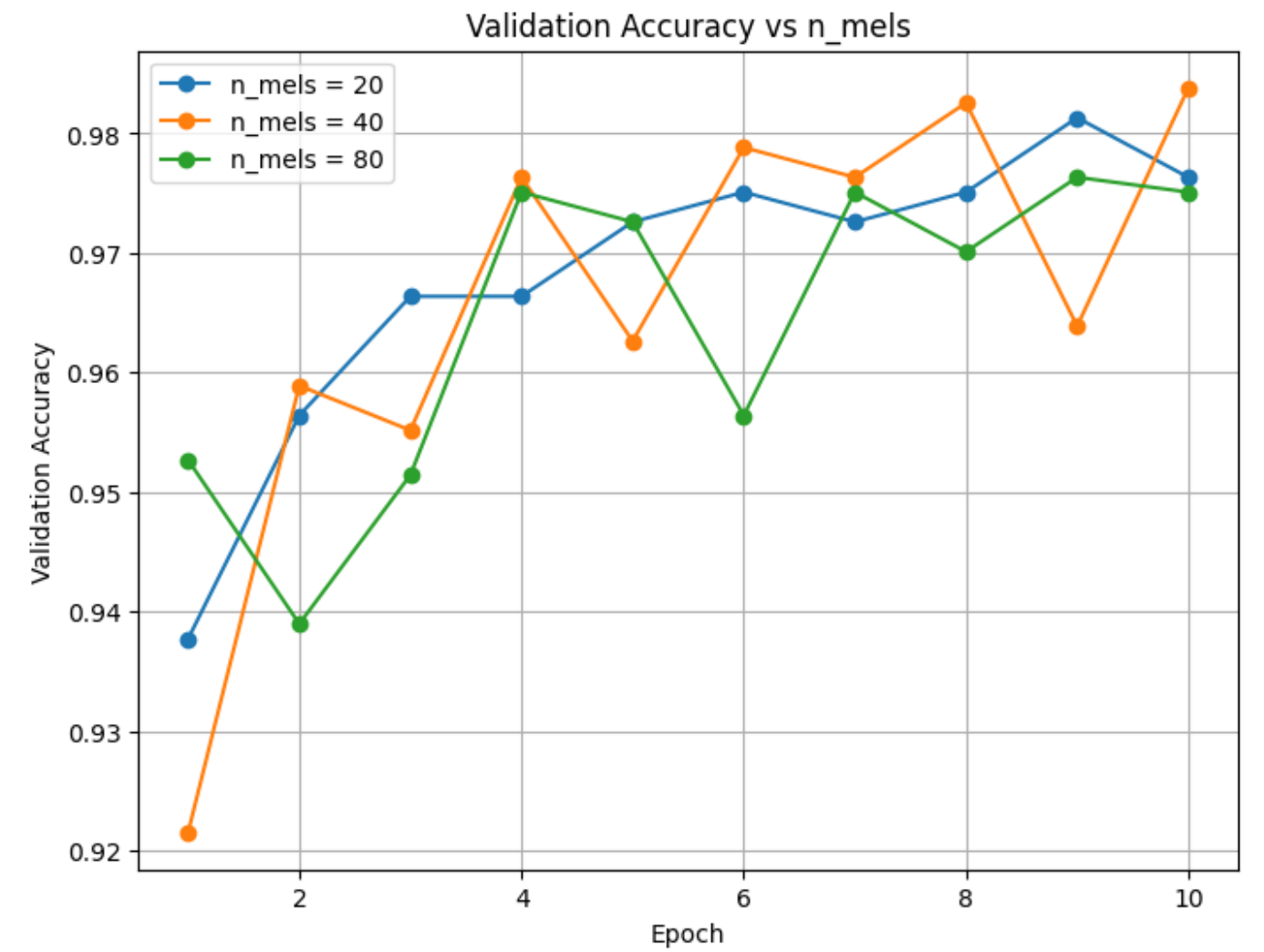
Validation loss also decreases with more Mel filters, particularly from 20 to 40. At `n_mels=80`, the loss fluctuates slightly, which may indicate some instability or overfitting.

Training Accuracy vs n_mels



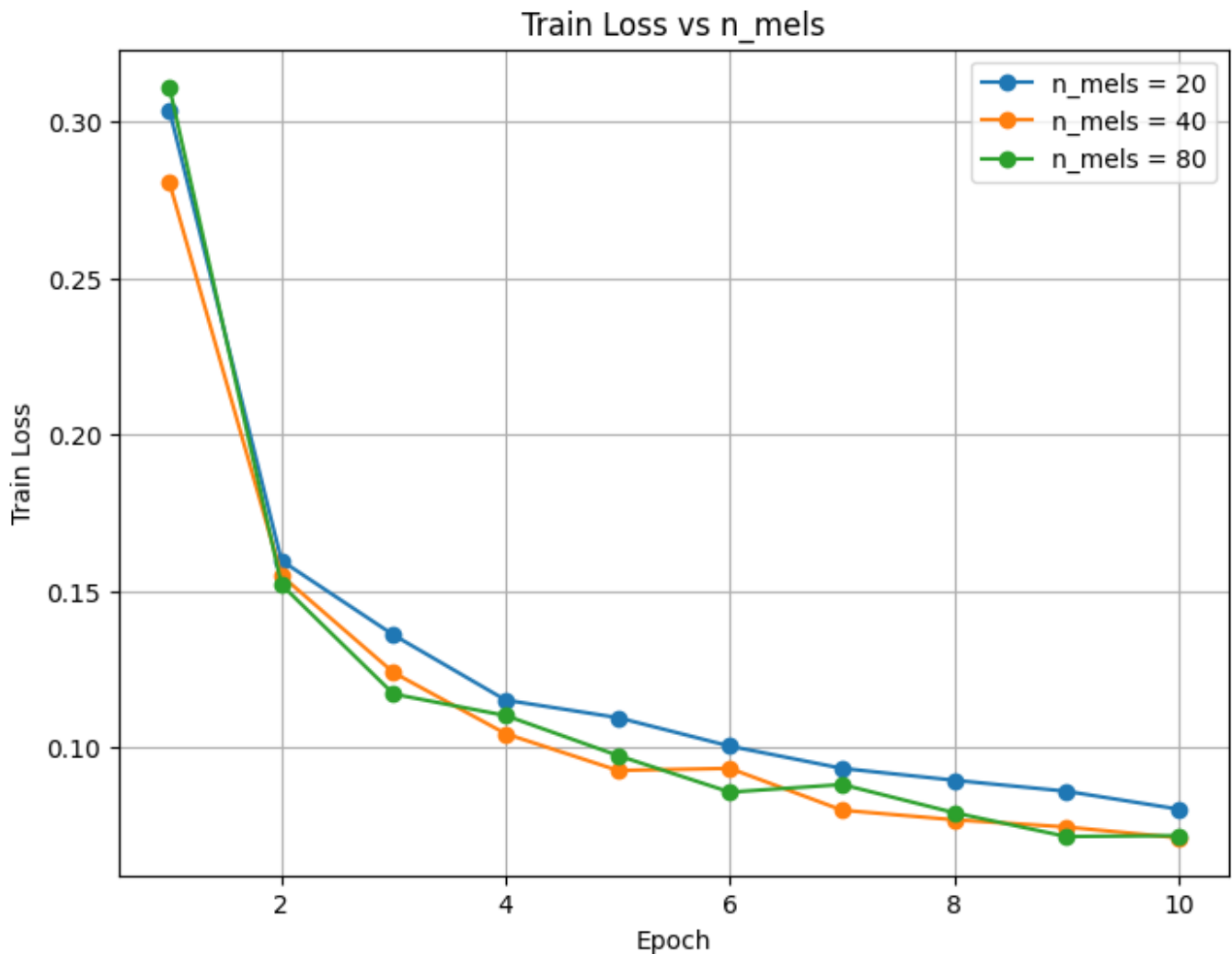
The training accuracy shows a consistent improvement with the number of Mel filters. A significant jump occurs between 20 and 40, followed by saturation beyond 40. This supports the idea that more detailed spectral representations help the model learn better.

Validation Accuracy vs n_mels



Validation accuracy follows a similar trend: improvement from 20 to 40 filters, then small fluctuations beyond that.

Training Time vs n_mels



As expected, the training time increases with the number of Mel filters.

From this analysis, I conclude that **40 Mel banks** is the most efficient configuration, offering high accuracy and relatively fast training. However, for consistency and further experiments with grouped convolutions, I selected **80 Mel banks as a baseline** for the next part of the assignment.

5.2. Experiments with Grouped Convolutions

To evaluate how the number of groups in the convolutional layer affects performance and efficiency, I conducted experiments with the following configurations:

- `groups = 2`
- `groups = 4`
- `groups = 8`
- `groups = 16`

For all experiments, the number of Mel filters (`n_mels`) was fixed at **80**, based on the findings from the previous section. Each configuration involved:

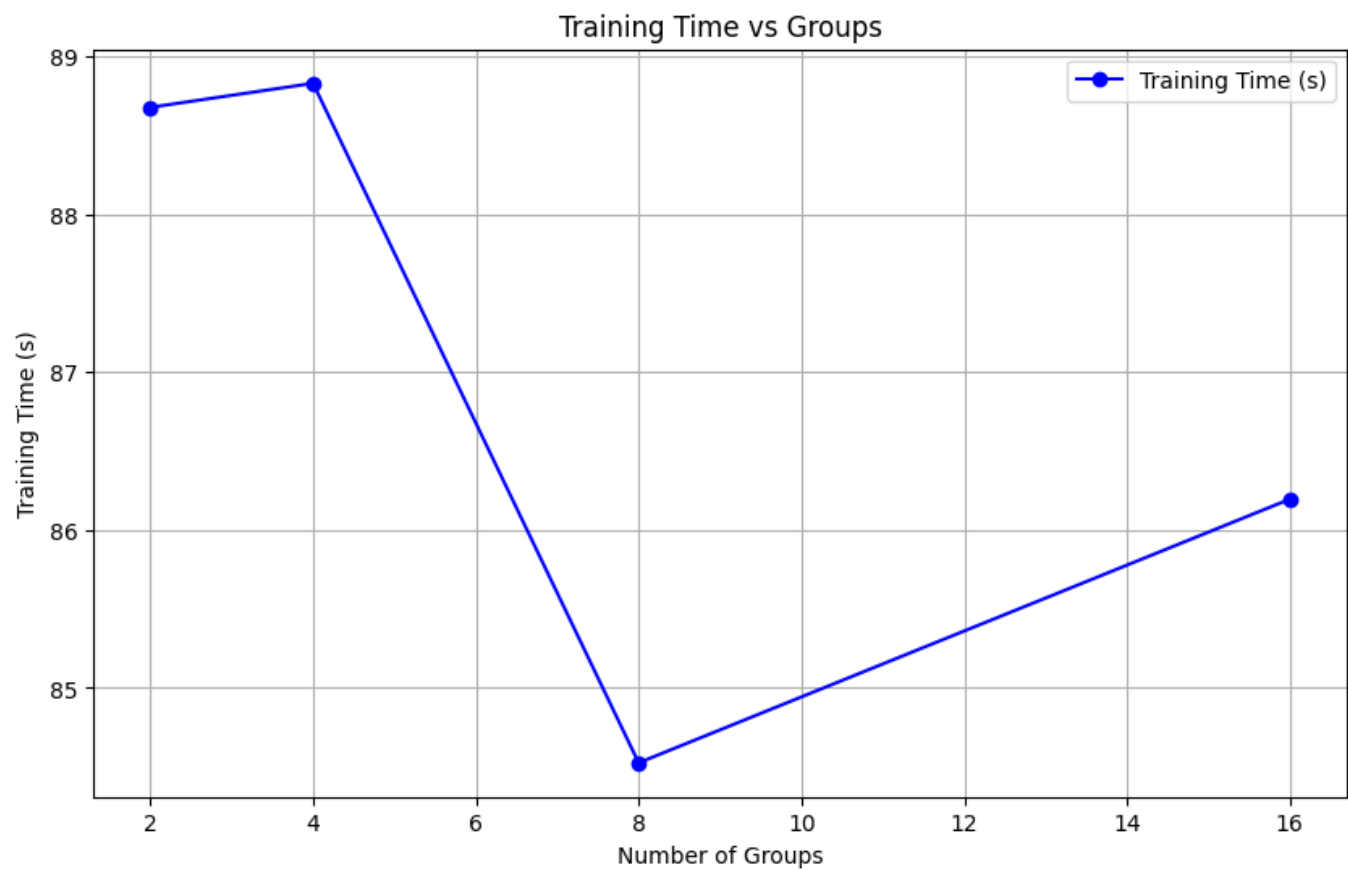
- Initializing the `SimpleCNN` model with a different number of groups in the `Conv1D` layer
- Training the model for 10 epochs
- Tracking training/validation loss and accuracy

- Measuring training time, number of parameters, and FLOPs

5.2.1. Graphs and Observations

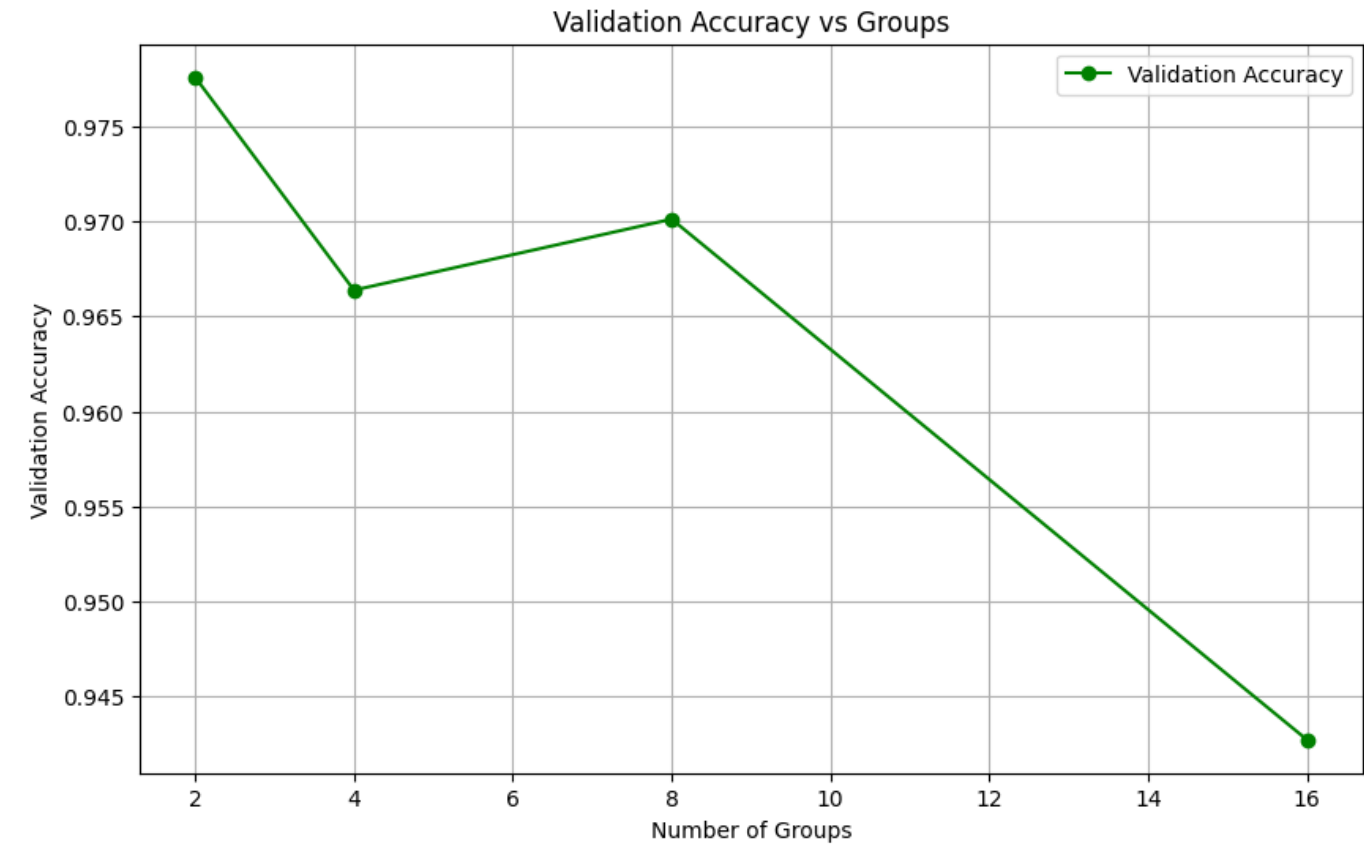
Below are the key plots that summarize how group convolutions influence model behavior and performance:

Training Time vs Groups



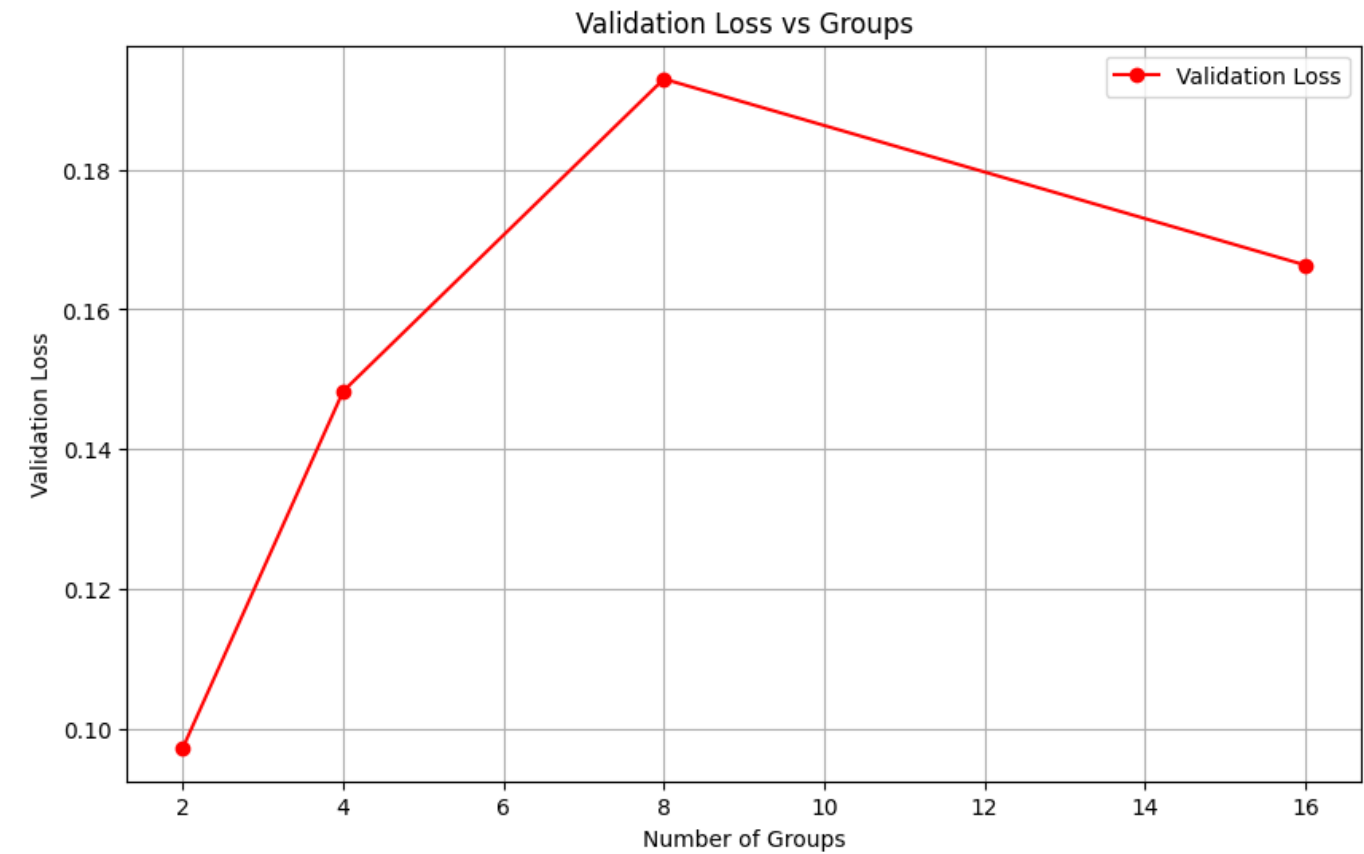
Training time noticeably decreases as the number of groups increases from 2 to 16. It is highest at 2 groups, drops significantly at 8 groups, and then slightly increases again at 16 groups. This could be related to how computations are distributed across groups in different configurations.

Validation Accuracy vs Groups



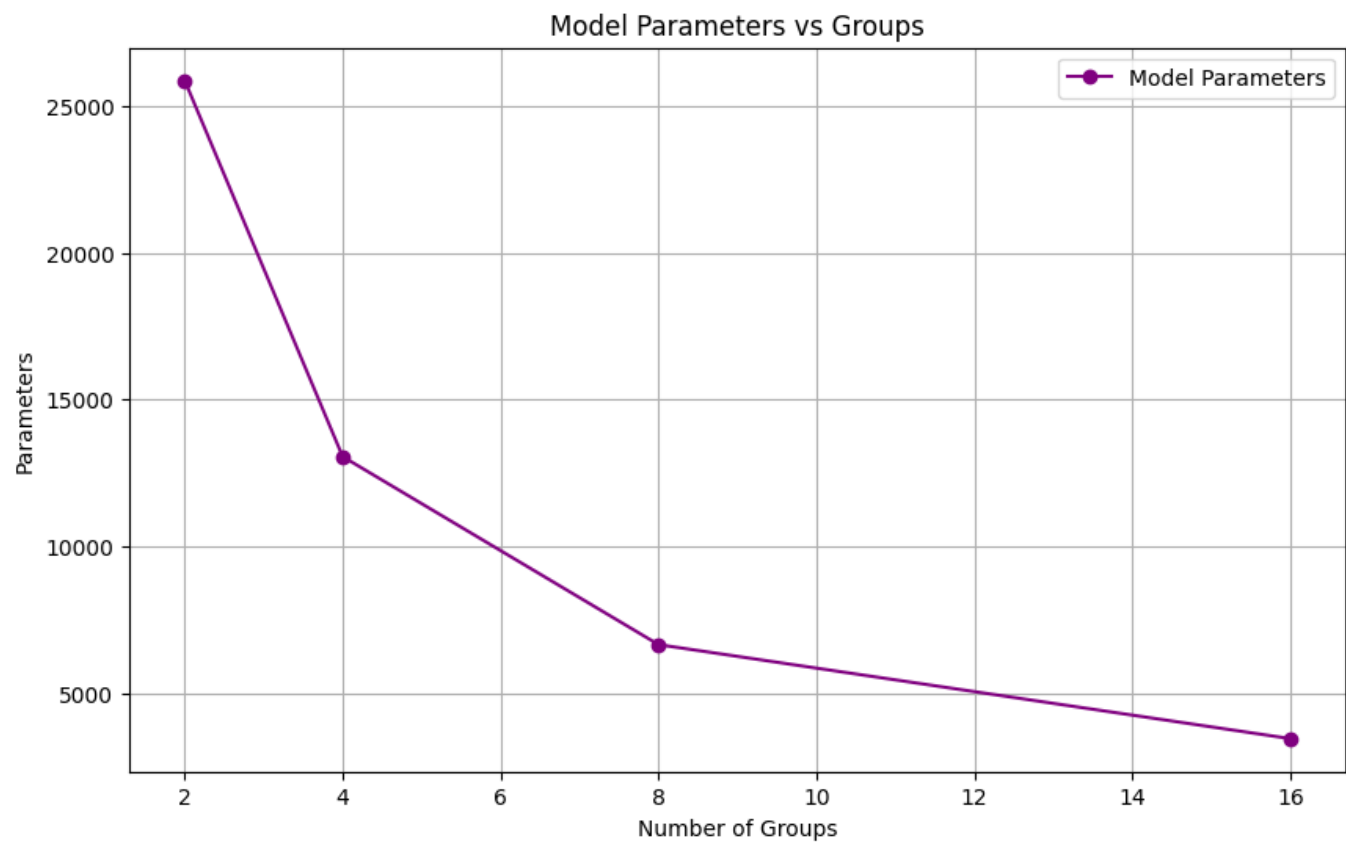
The highest validation accuracy is observed with 2 groups. Accuracy declines as the number of groups increases, suggesting that high group values may impair generalization. This may be due to reduced model complexity and fewer learnable parameters when groups are increased.

Validation Loss vs Groups



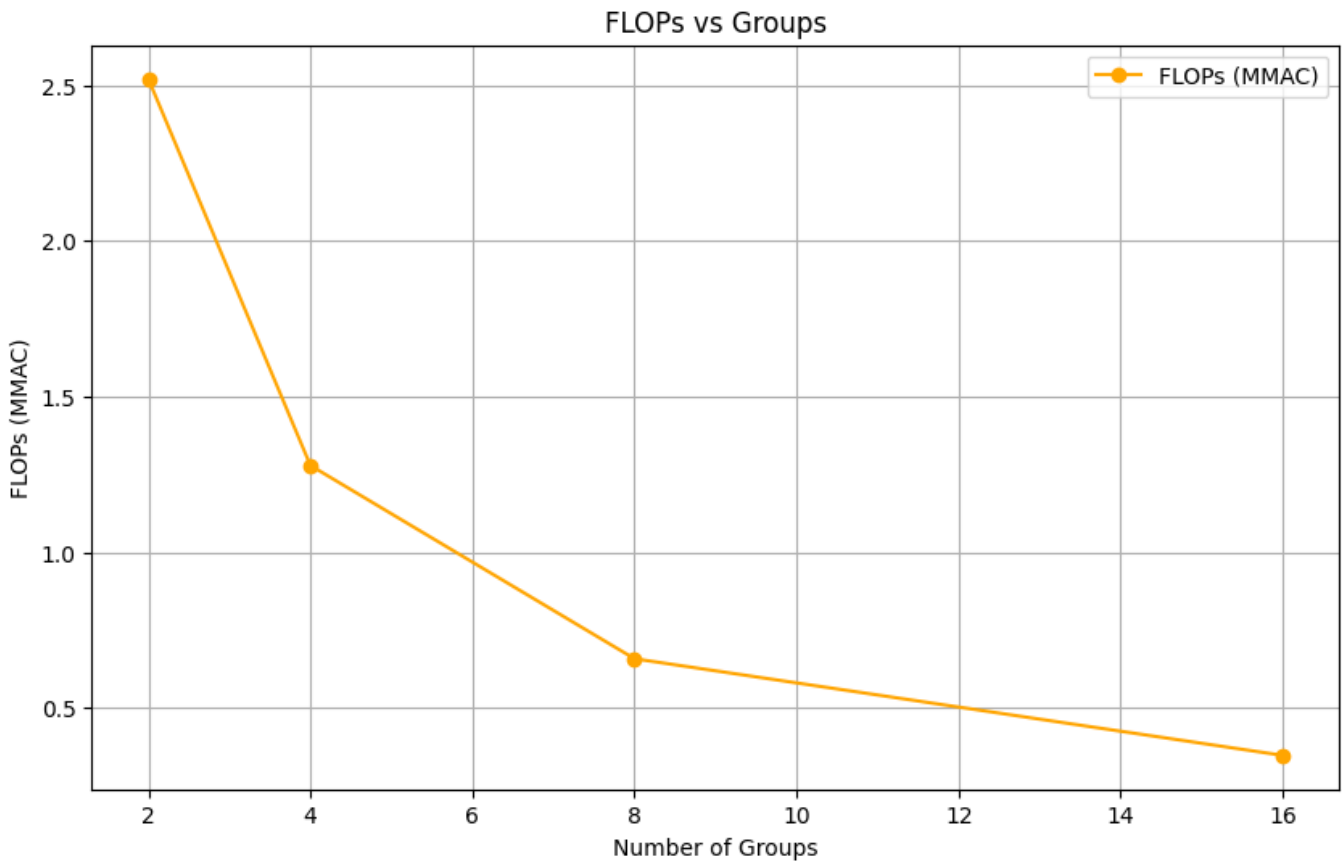
Validation loss increases as the number of groups grows, supporting the trend observed in the accuracy plot. This implies that increasing **groups** might harm model performance.

Model Parameters vs Groups



I observe a sharp drop in the number of model parameters as the number of groups increases. This happens because higher group values split the convolutional filters into smaller, independent sets, lowering the model's overall capacity.

FLOPs vs Groups



The number of floating-point operations also drops significantly as group size increases. Models with more groups are computationally cheaper — but this comes at the cost of reduced performance.

Conclusion

From this series of experiments, I conclude that smaller group values (2 or 4) yield the best performance in terms of validation accuracy.

Although larger groups reduce computation and model size, they also degrade performance. A trade-off exists between efficiency and accuracy, and for this task, **group=2** seems to be the sweet spot.

For the next section, I save the **group=2** model separately to compare it later against the baseline (**group=1**).

5.3. Baseline (Group = 1) vs Best Grouped Convolutions (Group = 2)

To evaluate how well the best model (Group = 2) generalizes, I compared it to the baseline (Group = 1) on the **test set**.

Both models were trained with `n_mels=80` and 10 epochs.

The results were as follows:

```
model_group1.pth: Test Loss = 0.0842, Accuracy = 0.9697 model_group2.pth: Test Loss = 0.6180,
Accuracy = 0.6529
```

The model with grouped convolutions (**Group = 2**) performed **significantly worse** on the test set. Despite showing the best validation results, it failed to generalize and likely **overfitted**.

Conclusion

The **baseline model (Group = 1)** clearly outperformed its grouped counterpart — achieving nearly **97% test accuracy** and **very low loss**.

This result suggests that the Group = 2 model might have **overfitted** or lacks sufficient capacity to generalize due to its **reduced number of parameters**.

In future experiments, it may be beneficial to test intermediate group sizes like **4 or 8**, to find a better trade-off between **model capacity** and **generalization performance**.

6. Key Observations

Experiment 1: Varying `n_mels`

- Higher `n_mels` reduced training and validation loss
- Accuracy improved up to `n_mels=40` then plateaued
- Training time and model complexity increased with `n_mels`
- Chose `n_mels=80` as baseline for further experiments

Experiment 2: Varying `groups`

- Validation accuracy peaked at `groups=2` but dropped beyond that
- Model parameters and FLOPs decreased with higher `groups`
- Training time had a non-linear pattern; best at `groups=8`

Experiment 3: Baseline vs Grouped Convs

- Despite performing best on validation, the grouped model (`groups=2`) **underperformed** on the test set
- Baseline (`groups=1`) had **much better generalization**, achieving ~97% accuracy

7. Conclusion

- **Best validation performance** was achieved with `groups=2, n_mels=80`
 - **Best generalization** (test set) was achieved with `groups=1`
 - Grouped convolutions reduce model complexity but may hurt generalization
-